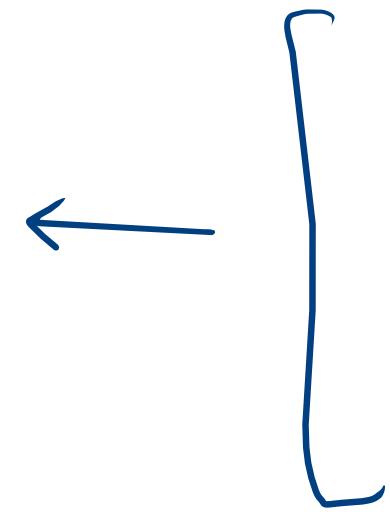
Agenda

1. Tensor flow & keras ✓
2. CNN ✓
3. Image representation
4. convolution operation
5. global pooling
6. visualizing feature maps & filters
7. Normalization

Tensor flow = engine

keras = Steering wheel.



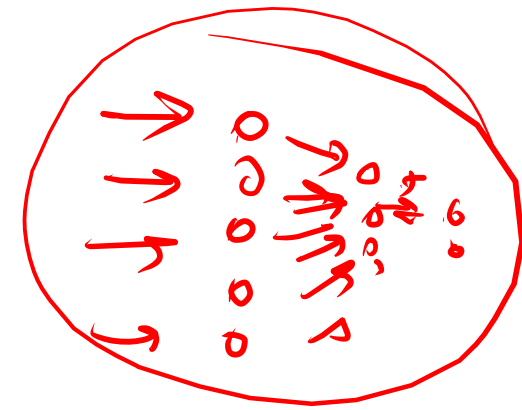
Tensor Flow

&

keras

data

Tensorflow → Open source Deep learning framework developed by ai.google.



Its job is to :-

→ Build neural n/w

→ Train models.

→ Calculate loss.

→ perform back propogation

→ update weights using optimizers

→ use CPU / GPU effectively

Spam detector → Not Spam
→ Spam

Tensor flow :

Input data

↓

Forward propogation

↓

Loss Calculation

↓

Back propogation

↓

Optimizers

↓

weight update

Forward propogation → process where input data passes through Neural N/w and each neuron calculate its o/p using weights, bias & activation

Back propogation → process of sending the error (loss) backward

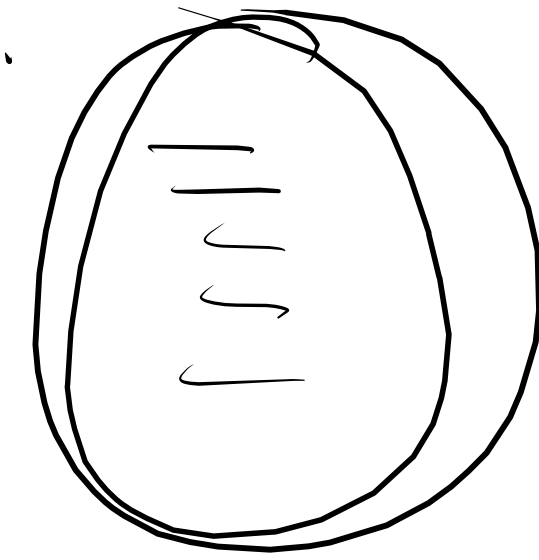
Tensor flow

Keras is a high level API that runs on top of the tensorflow.

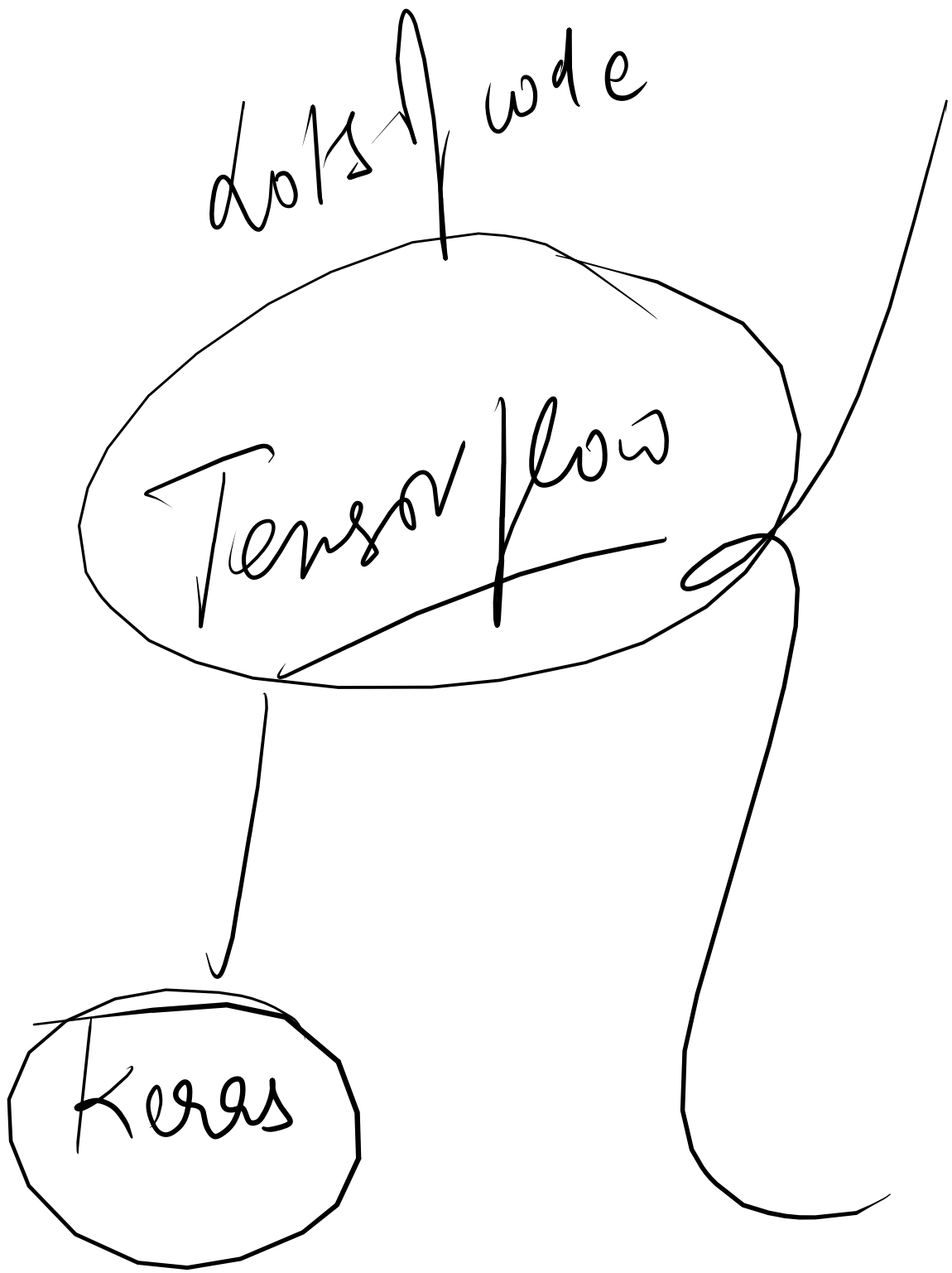
← Keras

Before keras,

- lots of Tensorflow code.
- lots of maths.
- lots of configuration.



with Keras, few simple lines.



Sigmoid

loss

gradient

updates weights

Repeat 1 until

Keras → Dense (10)

we are creating a layer of 10 neurons.

where each neuron receives i/p from all neurons in previous layer

df

	length
"win big"	8
"congts, you won iphone"	16
"welcome back"	12

0

0

1

0

1

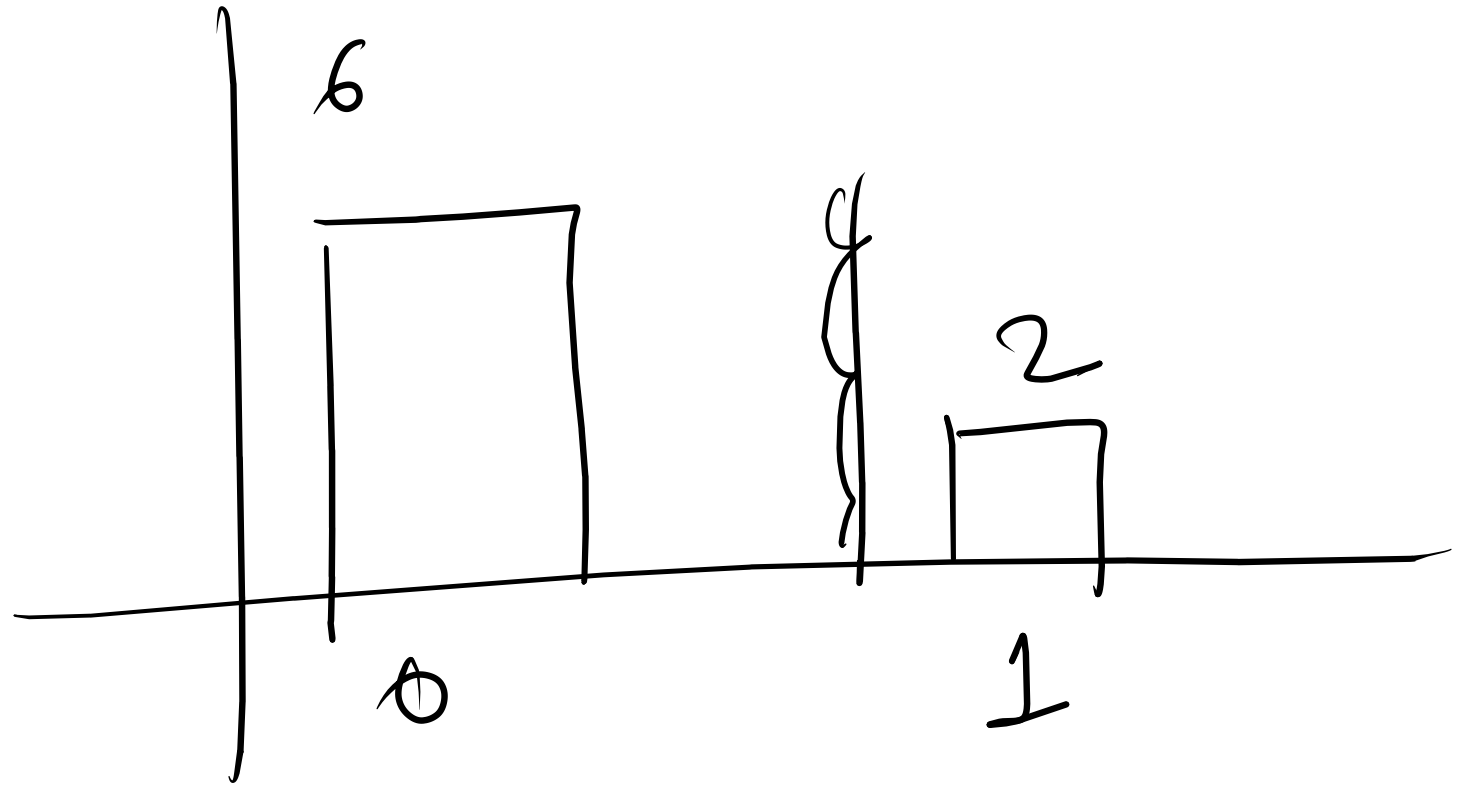
0

0

0

0 → HAM

1 → SPAM



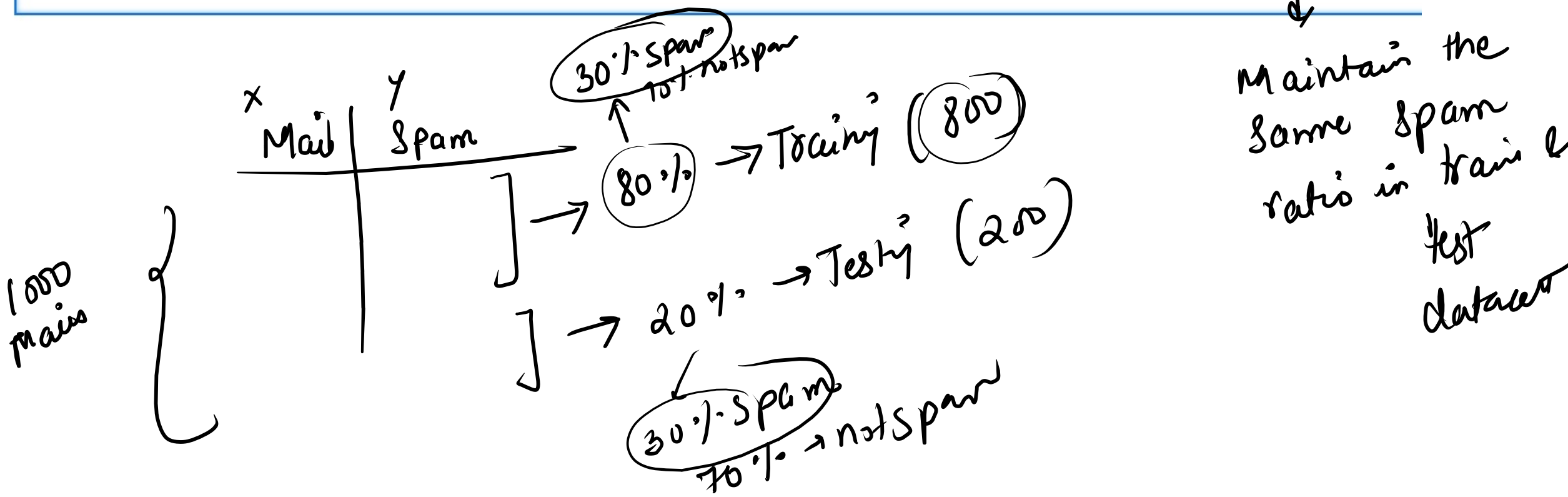
actual input messages.

output

```
X_train, X_test, y_train, y_test = train_test_split(
    padded, labels, test_size=0.2, random_state=42, stratify=labels
)

print(f"Train: {X_train.shape}  |  Test: {X_test.shape}")
print(f"Train spam rate: {y_train.mean():.2%}")
print(f"Test  spam rate: {y_test.mean():.2%}")
```

Maintain the same spam ratio in train & test datasets



```python
def build_model():
    model = Sequential([
        Embedding(VOCAB_SIZE, EMBEDDING_DIM, input_length=MAX_LEN),
        Bidirectional(LSTM(128, return_sequences=False)),
        Dropout(0.3),
        Dense(64, activation='relu'),
        Dropout(0.2),
        Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy',
                 tf.keras.metrics.Precision(name='precision'),
                 tf.keras.metrics.Recall(name='recall')]
    )
    return model

model = build_model()
model.summary()
```

Input Email

↓

Embedding layers

↓

Bidirectional LSTM

↓

Dropout (30%)

↓

Dense (64, ReLU)

↓

Dropout (20%)

activation
max (0, y)

Complete architecture

→ Cor — — — — iphone

←

SMS message → "congratulations, you won $5000. click here now"

↓

(Raw Text, Neural network cannot understand Raw text)

Cleaning → Code Removes — Special characters, URLs, Numbers, Extraspaces

↓

Tokenization → Tokenizer creates a dictionary

congratulations → 10
you → 5
won → 20

[10, 5, 20, 7, ...]

Now text is converted to number

↓

Padding → Every message must have same length.

[10, 5, 20, 7, ..., 0, 0, ...]

↓

Embedding

↓

BiLSTM → Brain of the model

congratulations → you → won → click → now

↓

Dropout (30%) → Keras

↓

Dense (64, ReLU)

↓

Dropout (20%)

↓

Dense (1, Sigmoid) ← final neuron

↓

Spam probability

end neuron output

Spam | Non Spam

0.80

0.20

100 neurons
30 neurons off
70 neurons on
Dropout 20% → 20% — off, 80% —

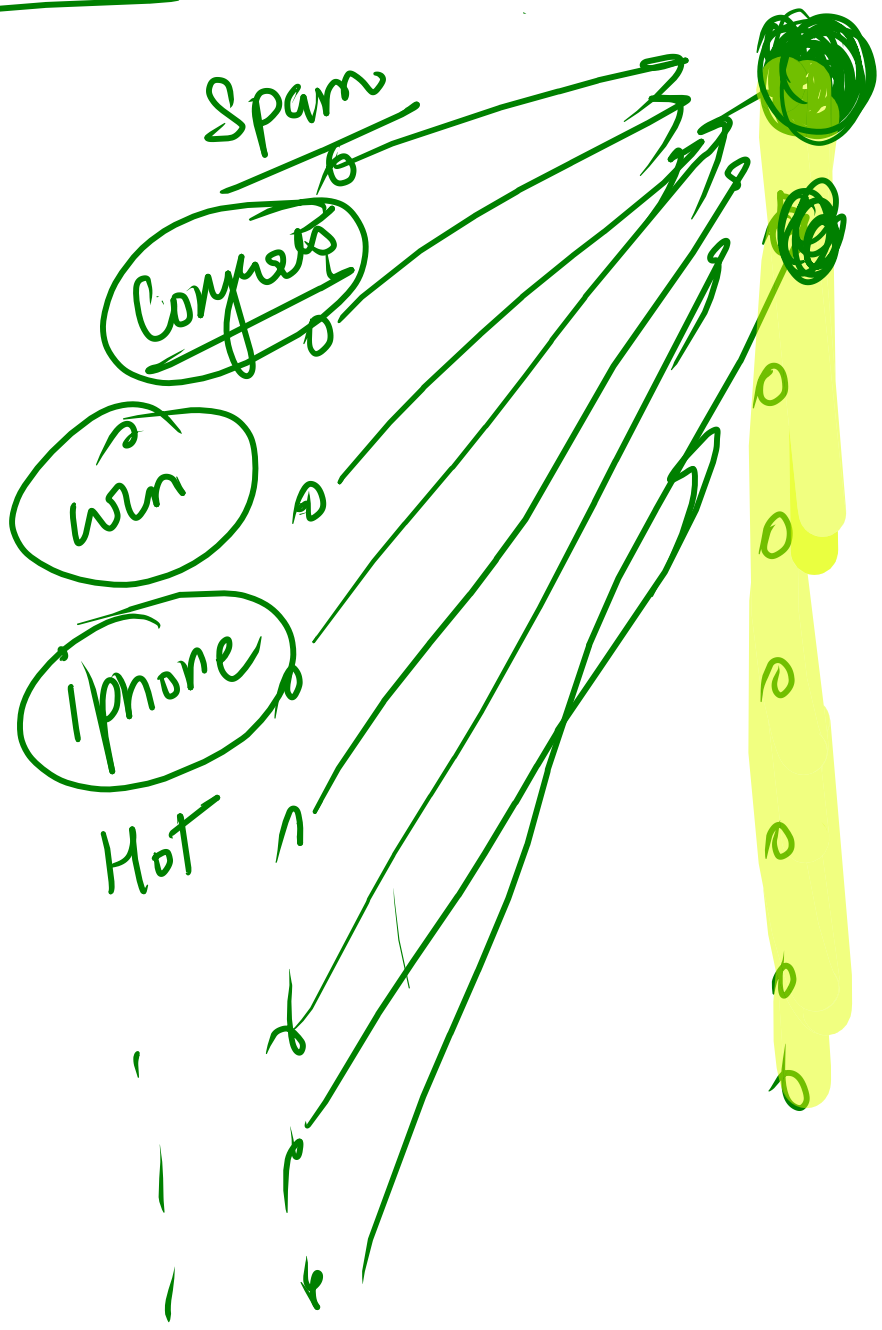
Combine pattern

Pattern A
won + prize

Pattern B
cash + click

Pattern: free + offer

Dense →



make combination

congrats, win

Keras

Dense

Image classification

Object classification

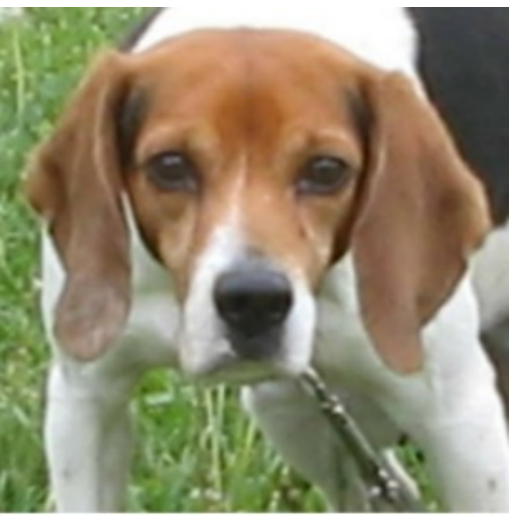
CNN

Convolutional Neural N/w

pixels

Face Recognition

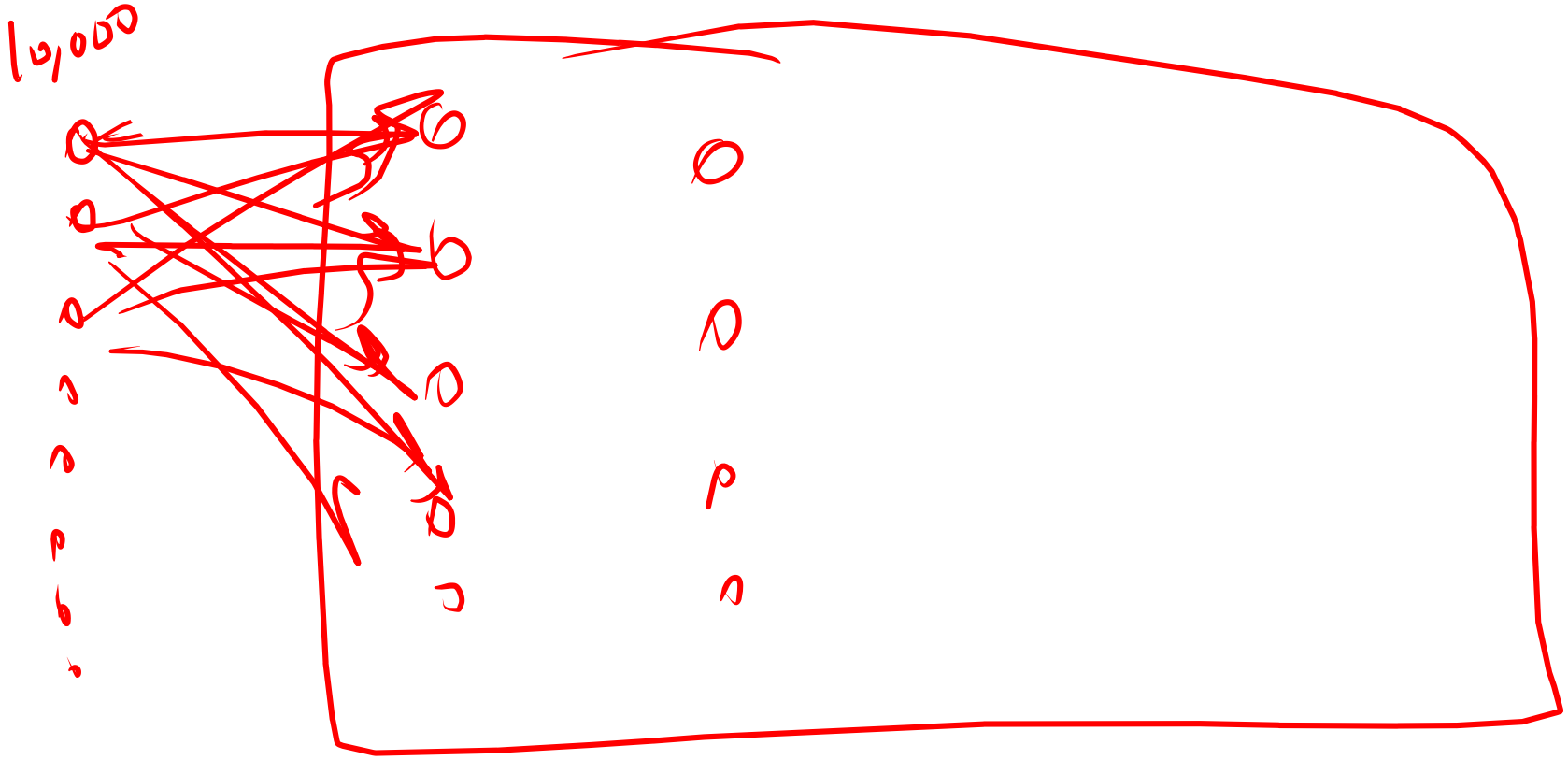
Medical Image analysis



100 x 100 pixels.

Total pixels

A traditional neural n/w would connect all 10,000 pixels to every neuron



Too many parameters

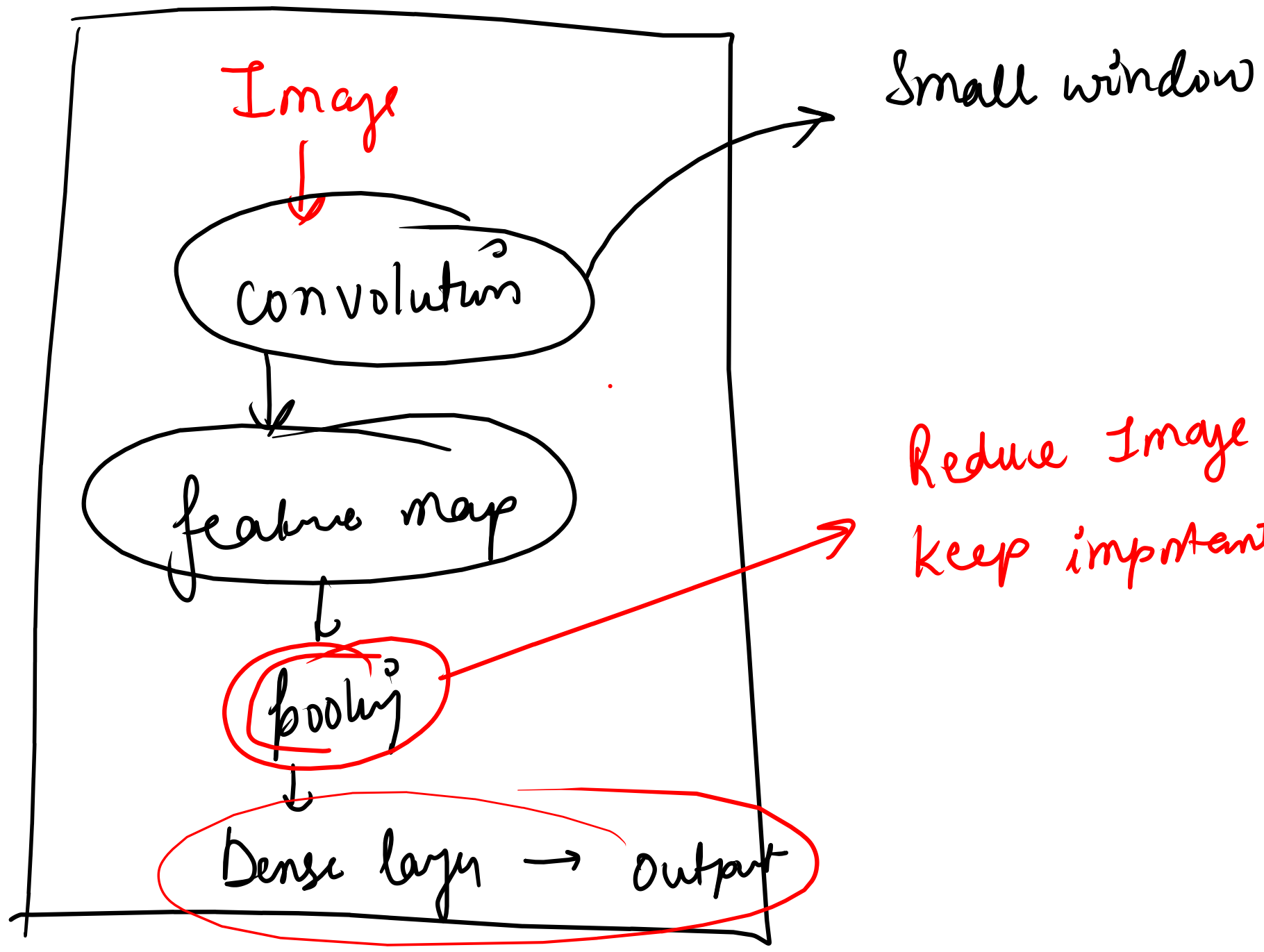
Slow training

High memory usage

Overfitting

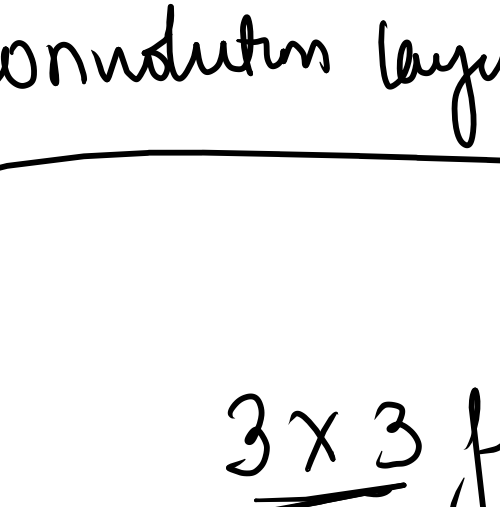
CNN

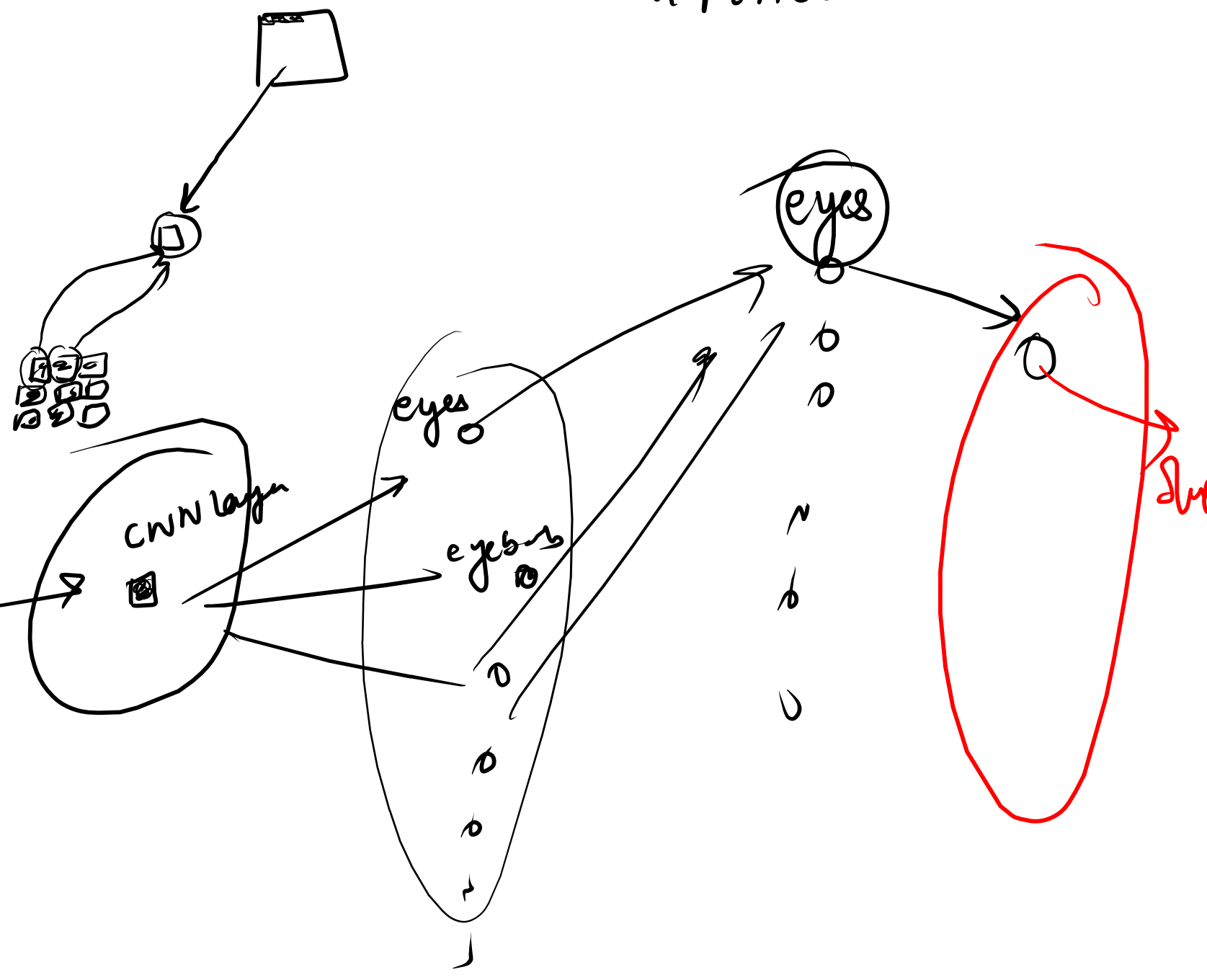
Instead of looking at the whole Image at once, CNN will look at small parts.



Convolution layer (CNN) → Imagine a small window called a filter

3 x 3 filter





Step3 → feature map → After convolution

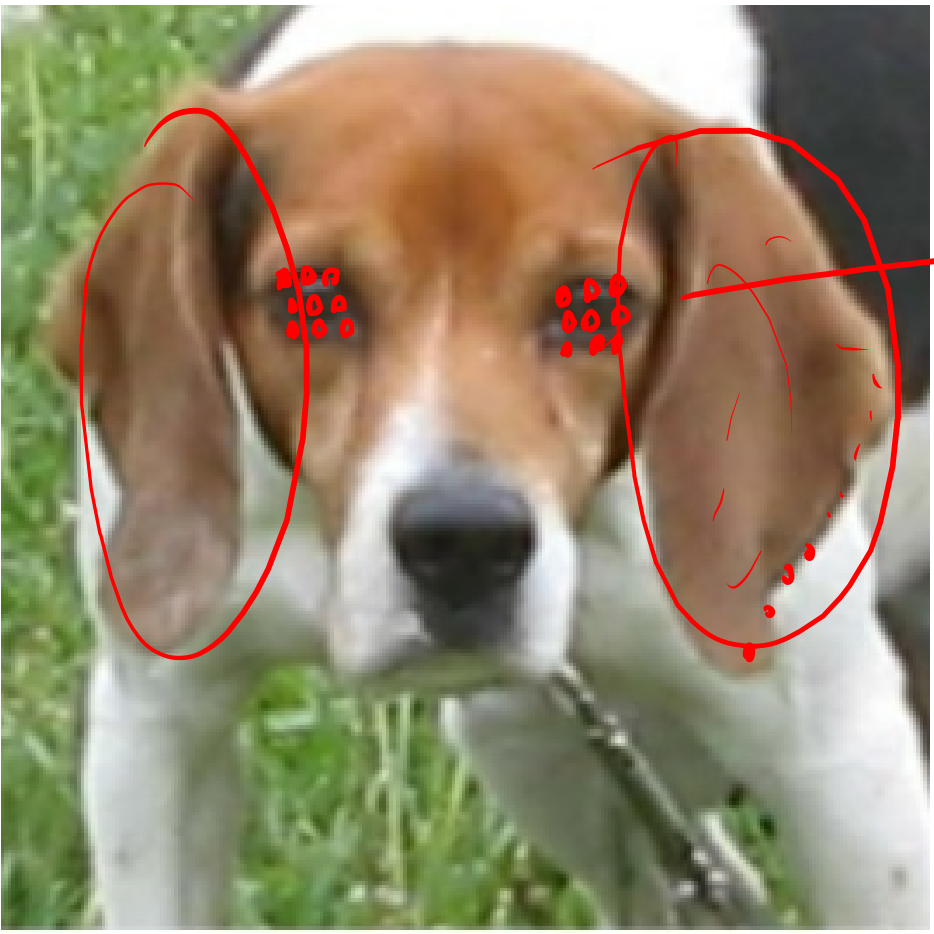
original Image

↓

convolution

↓

feature map.


eye
feature
convolution
layer
CNN

Scaling the pixel values to a small range so that the neural N/w can learn faster & more efficiently ← Normalization in CNN

(0 to 255) [255, 128, 64]

These large values can make training slower & less stable.

$\left[\frac{255}{255}, \frac{128}{255}, \frac{64}{255}\right]$

$= [1, 0.5, 0.4]$